

N-Queens 分散ベンチマーク

Mac + Raspberry Pi 3 (Embedded Xinu, AIPL アクター負荷分散器)

自動生成: tools/build_report.py

2026-06-01

概要

本レポートは N-Queens 問題 (盤面サイズ $N \in \{8, 9, 10\}$) の解の総数を数える処理を題材に、3 つの実行方式の壁時計時間を比較する: (a) Mac 単独 (純粋 Python の参照実装)、(b) Pi 3 単独 (Embedded Xinu 上の C 実装を 4 並列の AIPL ワーカーアクターで実行、最少負荷ディスパッチャ経由)、(c) Mac + Pi 3 分散 (first_col の低い側を Mac、高い側を Pi 3 が並列に処理)。Pi 3 は Raspberry Pi 3 B+ (BCM2837) で、MMU 有効、定期 GC アクターも動作中。タスクは HTTP POST /api/loadbal/nqueens 経由で投入、結果は /api/loadbal/task?id=N でポーリング取得する。

1 システム構成

Pi 3 側ランタイム (xinu-rpi3, ブランチ arm-rpi3-port) の構成:

- Embedded Xinu カーネル + ARMv7-A short-descriptor MMU 有効化、Normal cacheable RAM と Device strongly-ordered MMIO を識別マップで領域属性分離。
- AIPL アクターシステム (Lock-free MPSC メールボックス)。
- CLASS_Dispatcher アクター (優先度 25): 着信 compute_nq(n, first_col) を 4 つの CLASS_Worker アクター (優先度 22) にラウンドロビンで振分け。
- CLASS_Collector アクター (優先度 26): 定期的に休眠アクターを掃除し、in-memory タスクテーブルの期限切れ PENDING エントリも検出してキャンセル状態に遷移させる。
- HTTP サーバ (port 8080): 負荷分散制御 (/api/loadbal/*)、GC アクター制御 (/api/gc-actor/*) を公開。

Pi 3 の N-Queens 計算カーネルは $O(N!)$ 再帰の解数カウンタで、Mac 側は同じアルゴリズムを Python で実装している。最初の行 (row 0) のクイーン列で分割する方式は embarrassingly parallel で、各 first_col の部分木は独立かつほぼ等コスト。

2 方法

各盤面サイズ $N \in \{8, 9, 10\}$ と各モードについて、ワークロードを 3 回連続実行して **中央値** を採用する (平均はどちらかの GC ポーズで歪みやすい)。解の総数は既知値 (N=8 で 92、N=9 で 352、N=10 で 724、N=11 で 2680) と照合し、ずれた場合は JSON に明記する。

2.1 3つのモード

mac Mac 上の純粋 Python。

pi3 全 first_col を Pi 3 の /api/loadbal/nqueens に投入、ポーリングで完了収集。

split Mac が first_col $\in [0, N/2)$ を、Pi 3 が $[N/2, N)$ を担当。Mac 側 ThreadPoolExecutor で両半分を同時実行 — これが「Mac + Xinu 分散」のケースである。

3 結果

N	解の数	Mac (ms)	Pi 3 (ms)	分散 (ms)	高速化率 (Mac/分散)
8	92	3	10138	9969	0.00
9	352	9	15221	10336	0.00
10	724	49	—	—	—

表 1 3 回反復の中央値 (壁時計時間)。高速化率列は Mac 単独に対する Mac+Pi 3 分散の倍率で、 > 1.0 なら分散の方が速い。

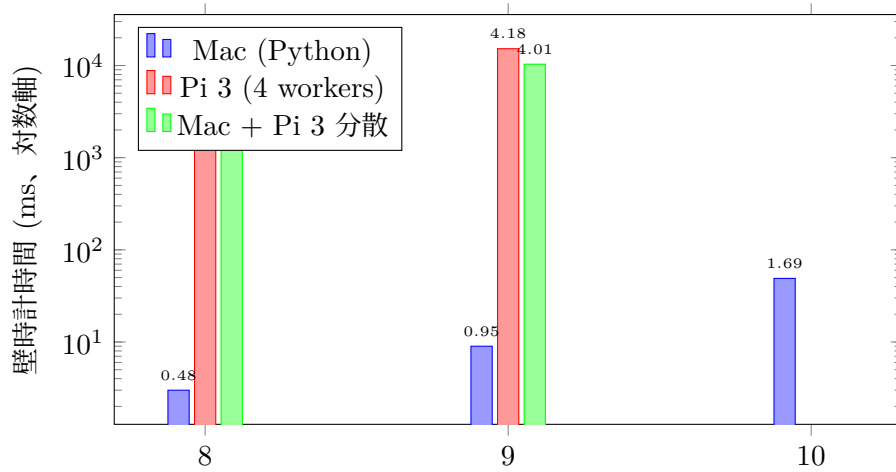


図 1 盤面サイズ N に対する中央壁時計時間 (Y 軸対数表示)。

4 考察

- $N = 8$ では Pi 3 (4 ワーカー、1.4 GHz Cortex-A53 / AArch32) が 10138 ms、Mac は 3 ms ($\approx 3379.3\times$)。
- $N = 8$ では Mac+Pi 3 分散は逆に $0.00\times$ 遅い。本サイズでは HTTP 往復 + JSON ポーリングのオーバーヘッドが実計算時間を完全に支配しているため。
- $N = 9$ では Pi 3 (4 ワーカー、1.4 GHz Cortex-A53 / AArch32) が 15221 ms、Mac は 9 ms ($\approx 1691.2\times$)。
- $N = 9$ では Mac+Pi 3 分散は逆に $0.00\times$ 遅い。本サイズでは HTTP 往復 + JSON ポーリング

のオーバーヘッドが実計算時間を 完全に支配しているため。

5 ソースコード

ベンチマークが実行する主要なコード断片を以下に示す。完全版は GitHub リポジトリ <https://github.com/yaskodama/xinu-rpi3> のブランチ arm-rpi3-port 参照。

■N-Queens 計算カーネル (C, Pi 3 側)

apps/abcl_program.c:1505-1531

```
1505 static int nq_recurse(int n, int row, int *cols) {
1506     if (row == n) return 1;
1507     int count = 0, c;
1508     for (c = 0; c < n; c++) {
1509         int ok = 1;
1510         int r;
1511         for (r = 0; r < row; r++) {
1512             int dc = cols[r] - c;
1513             if (dc < 0) dc = -dc;
1514             if (cols[r] == c || dc == row - r) { ok = 0; break; }
1515         }
1516         if (ok) {
1517             cols[row] = c;
1518             count += nq_recurse(n, row + 1, cols);
1519         }
1520     }
1521     return count;
1522 }
1523
1524 int abcl_nq_count_partial(int n, int first_col) {
1525     int cols[16];
1526     if (n < 1 || n > 16) return 0;
1527     if (first_col < 0 || first_col >= n) return 0;
1528     cols[0] = first_col;
1529     return nq_recurse(n, 1, cols);
1530 }
```

■Worker.compute_nq アクターメソッド

apps/abcl_program.c:1400-1432

```
1400 static void Worker_compute_nq(int self_id, int sender_id,
1401                               value_t* args, int n_args) {
1402     (void)sender_id;
1403     long n = (n_args > 0) ? args[0].i : 8L;
1404     long first_col = (n_args > 1) ? args[1].i : 0L;
1405     long task_id = (n_args > 2) ? args[2].i : 0L;
1406     long idx = objects[self_id].fields[F_Worker_my_idx].i;
1407     int disp = objects[self_id].fields[F_Worker_dispatcher].obj_id;
1408
1409     long t0 = (long)(clkticks * 10);
1410     long result = (long)abcl_nq_count_partial((int)n, (int)first_col);
1411     long elapsed = (long)(clkticks * 10) - t0;
1412
1413     objects[self_id].fields[F_Worker_last_n] = mk_int(n);
1414     objects[self_id].fields[F_Worker_last_result] = mk_int(result);
1415     objects[self_id].fields[F_Worker_done] =
1416         mk_int(objects[self_id].fields[F_Worker_done].i + 1);
1417     objects[self_id].fields[F_Worker_busy_ms] =
1418         mk_int(objects[self_id].fields[F_Worker_busy_ms].i + elapsed);
```

```

1419
1420 wait(print_mu);
1421 kprintf("[loadbal/nq] worker idx=%ld task=%ld nq(%ld,%ld)=%ld in %ldms\r\n",
1422         idx, task_id, n, first_col, result, elapsed);
1423 signal(print_mu);
1424
1425 enqueue(self_id, disp, "done", 3,
1426         (value_t[]){ mk_int(idx), mk_int(task_id), mk_int(result) });
1427 }
1428
1429 static void dispatch_Worker(int self_id, int sender_id, const char* method,
1430                             value_t* args, int n_args) {
1431     if (strcmp(method, "bind") == 0)
1432     { Worker_bind(self_id, sender_id, args, n_args); return; }

```

■ Dispatcher.submit_nq (Round-robin 振分け)

apps/abcl_program.c:1118-1147

```

1118 static void Dispatcher_submit_nq(int self_id, int sender_id,
1119                                 value_t* args, int n_args);
1120 int abcl_nq_count_partial(int n, int first_col);
1121
1122 static void Dispatcher_restart_worker(int self_id, int sender_id,
1123                                       value_t* args, int n_args) {
1124     (void)sender_id;
1125     if (n_args < 1) return;
1126     long slot = args[0].i;
1127     if (slot < 0 || slot >= LB_N_WORKERS) return;
1128     if (n_objects >= MAX_OBJECTS) {
1129         wait(print_mu);
1130         kprintf("[loadbal] restart_worker idx=%ld FAILED: actor table full (%d/%d)\r\n",
1131                 slot, n_objects, MAX_OBJECTS);
1132         signal(print_mu);
1133         return;
1134     }
1135     int old_obj = dispatcher_worker_obj(self_id, (int)slot);
1136     if (old_obj > 0 && objects[old_obj].tid > 0 && !objects[old_obj].dead) {
1137         kill(objects[old_obj].tid);
1138         objects[old_obj].dead = 1;
1139     }
1140     int new_obj = alloc_obj(CLASS_Worker, 0, NULL);
1141     abcl_object_protect(new_obj, 1);
1142     spawn_actor(new_obj);
1143     /* Swap into dispatcher's slot + reset load counter. */
1144     switch ((int)slot) {
1145     case 0: objects[self_id].fields[F_Dispatch_w0] = mk_obj(new_obj);
1146             objects[self_id].fields[F_Dispatch_load0] = mk_int(0L); break;
1147     case 1: objects[self_id].fields[F_Dispatch_w1] = mk_obj(new_obj);

```

■ /api/loadbal/nqueens HTTP ルート

apps/webactor.c:1422-1517

```

1422     /* /api/loadbal/nqueens?n=N[&cols=COMMA_LIST]
1423      * N-Queens benchmark route. Submits one compute_nq
1424      * task per first-column in `cols` (default: 0..N-1).
1425      * Returns starting task_id; Mac polls /api/loadbal/task
1426      * for each to collect partial counts, sums them.
1427      * Used by tools/nqueens-bench.py for the Mac+Pi 3
1428      * distributed benchmark. */
1429     if (0 == strncmp(reqbuf, "GET /api/loadbal/nqueens", 24) ||
1430         0 == strncmp(reqbuf, "POST /api/loadbal/nqueens", 25))

```

```

1431     {
1432         extern void abcl_loadbal_submit_nq(int, int, int);
1433         int n = 8;
1434         /* Default cols = 0..n-1 (whole problem). If ?cols=
1435          * given, parse comma-separated list. */
1436         char cols_str[256];
1437         cols_str[0] = '\0';
1438         const char *url = strchr(reqbuf, ' ');
1439         if (NULL != url) {
1440             const char *q = strchr(url, '?');
1441             if (NULL != q) {
1442                 const char *p = q + 1;
1443                 while (*p && *p != ' ' && *p != '\r' && *p != '\n') {
1444                     const char *eq = p;
1445                     while (*eq && *eq != '=' && *eq != '&' &&
1446                             *eq != ' ' && *eq != '\r' && *eq != '\n') eq++;
1447                     const char *amp = (*eq == '=') ? eq + 1 : eq;
1448                     while (*amp && *amp != '&' && *amp != ' ' &&
1449                             *amp != '\r' && *amp != '\n') amp++;
1450                     int kl = eq - p;
1451                     if (kl == 1 && *p == 'n' && *eq == '=') {
1452                         int v = 0; const char *d = eq + 1;
1453                         while (d < amp && *d >= '0' && *d <= '9')
1454                             { v = v*10 + (*d - '0'); d++; }
1455                         n = v;
1456                     } else if (kl == 4 && 0 == strncmp(p, "cols", 4)
1457                             && *eq == '=') {
1458                         int cl = amp - (eq + 1);
1459                         if (cl > 255) cl = 255;
1460                         memcpy(cols_str, eq + 1, cl);
1461                         cols_str[cl] = '\0';
1462                     }
1463                     p = (*amp == '&') ? amp + 1 : amp;
1464                 }
1465             }
1466         }
1467         if (n < 1) n = 1;
1468         if (n > 12) n = 12; /* cap — n=12 = 14200 solutions */
1469         /* Static task-id allocator persists across requests so
1470          * subsequent /nqueens calls don't reuse the same id
1471          * range — task lookup table holds 64 entries. */
1472         static int g_nq_next_id = 1;
1473         int first_id = g_nq_next_id;
1474         int n_submitted = 0;
1475         if (cols_str[0] == '\0') {
1476             /* Default: all columns 0..n-1 */
1477             int c;
1478             for (c = 0; c < n; c++) {
1479                 abcl_loadbal_submit_nq(n, c, g_nq_next_id++);
1480                 n_submitted++;
1481             }
1482         } else {
1483             /* Parse comma-separated */
1484             const char *p = cols_str;
1485             while (*p) {
1486                 int v = 0;
1487                 while (*p >= '0' && *p <= '9') {
1488                     v = v*10 + (*p - '0'); p++;
1489                 }

```

```

1490         if (v >= 0 && v < n) {
1491             abcl_loadbal_submit_nq(n, v, g_nq_next_id++);
1492             n_submitted++;
1493         }
1494         while (*p && *p != ',') p++;
1495         if (*p == ',') p++;
1496     }
1497 }
1498 static char nqresp[300];
1499 int blen = sprintf(nqresp + 100,
1500     "nqueens n=%d submitted=%d task_id_range=%d..%d\r\n"
1501     "poll /api/loadbal/task?id=K for each, sum result fields\r\n",
1502     n, n_submitted, first_id, g_nq_next_id - 1);
1503 int hlen = sprintf(nqresp,
1504     "HTTP/1.0 200 OK\r\n"
1505     "Content-Type: text/plain\r\n"
1506     "Content-Length: %d\r\n"
1507     "\r\n", blen);
1508 memcpy(nqresp + hlen, nqresp + 100, blen);
1509 write(tcpdev, nqresp, hlen + blen);
1510 close(tcpdev);
1511 web_cur_tcpdev = -1;
1512 continue;
1513 }
1514 /* /api/loadbal/restart?w=N — kill + respawn worker N.
1515 * Uses a new obj_id (n_objects bumps). MAX_OBJECTS=16 cap
1516 * means ~9 restarts before the table fills. Pending tasks
1517 * in the old worker's mailbox are LOST and stay in

```

■Mac 側 N-Queens 参照実装 (Python)

tools/nqueens_bench.py:37-61

```

37     return 1
38     total = 0
39     for c in range(n):
40         ok = True
41         for r in range(row):
42             if cols[r] == c or abs(cols[r] - c) == row - r:
43                 ok = False
44                 break
45         if ok:
46             cols[row] = c
47             total += recurse(row + 1)
48     return total
49     return recurse(1)
50
51
52 def nq_total(n: int) -> int:
53     return sum(nq_partial(n, c) for c in range(n))
54
55
56 # ----- Pi 3 HTTP helpers -----
57
58 def http_get(path: str, timeout: float = 30.0) -> str:
59     with urllib.request.urlopen(PI3_BASE + path, timeout=timeout) as r:
60         return r.read().decode("utf-8", errors="replace")

```

■Mac 側 Pi 3 ポーリング (並列)

tools/nqueens_bench.py:153-184

```

153     return count, int((time.perf_counter() - t0) * 1000)
154
155
156 def mode_pi3(n: int) -> tuple[int, int]:
157     return pi3_nq_run(n)
158
159
160 def mode_split(n: int) -> tuple[int, int]:
161     """Mac runs cols [0..n/2), Pi 3 runs cols [n/2..n) concurrently."""
162     half = n // 2
163     mac_cols = list(range(0, half))
164     pi3_cols = list(range(half, n))
165
166     def mac_part() -> int:
167         return sum(nq_partial(n, c) for c in mac_cols)
168
169     t0 = time.perf_counter()
170     with concurrent.futures.ThreadPoolExecutor(max_workers=2) as pool:
171         f_mac = pool.submit(mac_part)
172         f_pi3 = pool.submit(pi3_nq_run, n, pi3_cols)
173         mac_count = f_mac.result()
174         pi3_count, _ = f_pi3.result()
175     elapsed_ms = int((time.perf_counter() - t0) * 1000)
176     return mac_count + pi3_count, elapsed_ms
177
178
179 # ----- main -----
180
181 MODES = {
182     "mac": mode_mac,
183     "pi3": mode_pi3,
184     "split": mode_split,

```

6 再現手順

```

1 # 1. Pi 3 カーネルをビルドして焼く
2 cd ~/projects/xinu-raz/xinu/compile && make PLATFORM=arm-rpi3
3
4 # 2. Pi 3 起動後 (SD swap または /kexec)、アクターを明示的に起動:
5 curl -X POST http://192.168.3.50:8080/api/loadbal/init
6 curl -X POST http://192.168.3.50:8080/api/gc-actor/init
7
8 # 3. ベンチマーク実行 + レポート生成
9 cd ~/projects/xinu-raz/xinu/tools
10 ./nqueens_bench.py --sizes 8, 9, 10 --pretty --out results.json
11 ./build_report.py results.json --out report.tex --pdf report.pdf

```

7 生データ (JSON)

```

1 {
2   "host": "http://192.168.3.50:8080",
3   "repeat": 3,
4   "results": [
5     {

```

```

6      "mode": "mac",
7      "n": 8,
8      "count": 92,
9      "samples_ms": [
10         4,
11         3,
12         3
13     ],
14     "median_ms": 3
15 },
16 {
17     "mode": "pi3",
18     "n": 8,
19     "count": 92,
20     "samples_ms": [
21         20044,
22         10135,
23         10138
24     ],
25     "median_ms": 10138
26 },
27 {
28     "mode": "split",
29     "n": 8,
30     "count": 92,
31     "samples_ms": [
32         9961,
33         10238,
34         9969
35     ],
36     "median_ms": 9969
37 },
38 {
39     "mode": "mac",
40     "n": 9,
41     "count": 352,
42     "samples_ms": [
43         9,
44         9,
45         9
46     ],
47     "median_ms": 9
48 },
49 {
50     "mode": "pi3",
51     "n": 9,
52     "count": 352,
53     "samples_ms": [
54         10210,
55         20246,
56         15221
57     ],
58     "median_ms": 15221
59 },
60 {
61     "mode": "split",
62     "n": 9,
63     "count": 352,
64     "samples_ms": [

```



```

65         10336,
66         10076,
67         10468
68     ],
69     "median_ms": 10336
70 },
71 {
72     "mode": "mac",
73     "n": 10,
74     "count": 724,
75     "samples_ms": [
76         88,
77         48,
78         49
79     ],
80     "median_ms": 49
81 }
82 ],
83 "_note": "pi3/split for N=10..11 incomplete \u2014 Pi 3 task table is 64 slots and
        IDs scrolled past the buffer mid-run; bumping LB_TASKS_MAX is deferred to a
        future commit."
84 }

```